

Implementación en Python de la Arquitectura Óptima de Red Neuronal Profunda para la Estimación y Predicción del SOC.

Estimación del Carbono Orgánico del Suelo mediante Redes Neuronales de Grafos utilizando Datos Multimodales Geoespaciales

Director del proyecto: David Andrés Donoso Vargas

Servidor Público 5: Mateo Xavier Soliz Villafuerte

Fecha de entrega: 04 de marzo de 2026

Tabla de contenido

1.Introducción	3
2.Objetivos	4
3.Metodología	5
4.Conclusiones	27
6.Bibliografía	28
7.Anexos	29

1. Introducción

El presente informe describe el proceso de implementación en Python de arquitecturas avanzadas de aprendizaje profundo orientadas a la estimación y predicción del Carbono Orgánico del Suelo (SOC, por sus siglas en inglés), utilizando enfoques basados en Redes Neuronales de Grafos (GNN, por sus siglas en inglés) y sus variantes para el modelado del SOC, considerando las relaciones espaciales entre observaciones geográficas. El trabajo se enmarca en las metodologías modernas de mapeo digital de suelos (*Digital Soil Mapping*, DSM), cuyo objetivo es generar estimaciones espaciales continuas de propiedades del suelo a partir de datos puntuales de muestreo y variables ambientales auxiliares.

El propósito central de este proyecto es estimar la cantidad de SOC muestreado en 2021 en Ecuador mediante la integración de múltiples fuentes de información ambiental y geoespacial. En particular, se busca combinar datos provenientes de muestras de suelo obtenidas in situ, disponibles en fuentes oficiales, con variables ambientales que describen los principales factores de formación del suelo, tales como precipitación, temperatura, humedad, variables topográficas y atributos derivados de imágenes satelitales provenientes de los sensores *Sentinel-1*, *Sentinel-2* y *Sentinel-3*. Este enfoque permite aprovechar la complementariedad entre mediciones directas del suelo y variables ambientales de cobertura continua, facilitando la construcción de modelos predictivos capaces de extrapolar información hacia áreas donde no existen observaciones directas.

Dentro de este contexto, el informe establece una base técnica para el desarrollo de modelos predictivos de SOC, mediante la caracterización e implementación de arquitecturas de aprendizaje profundo capaces de modelar dependencias espaciales complejas. En particular, se evalúan diferentes enfoques basados en GNN, las cuales permiten representar explícitamente las relaciones de proximidad entre puntos de muestreo mediante estructuras de grafo. A diferencia de los métodos tradicionales de aprendizaje automático, que suelen asumir independencia entre observaciones, las GNN incorporan directamente la estructura espacial del problema, capturando interacciones entre muestras cercanas que pueden influir en la distribución del SOC.

En el marco de este estudio se implementan en el lenguaje de programación de Python diferentes arquitecturas de redes neuronales profundas orientadas a problemas de regresión sobre nodos espaciales, específicamente *Graph Attention Networks* (GATv2) y

Graph Sample and Aggregate (GraphSAGE). Estas arquitecturas representan enfoques complementarios dentro del aprendizaje sobre grafos. Por un lado, los modelos basados en mecanismos de atención, como GATv2, asignan pesos adaptativos a los nodos vecinos, identificando cuáles contribuyen de manera más significativa a la estimación del SOC del nodo objetivo. Por otro lado, los modelos basados en agregación y muestreo, como GraphSAGE, generan representaciones de nodos a partir de la información de sus vecinos, lo que facilita la generalización hacia nuevas regiones geográficas no observadas durante el entrenamiento.

Adicionalmente, el estudio compara dos esquemas experimentales fundamentales en el aprendizaje sobre grafos: i) el aprendizaje transductivo y ii) el aprendizaje inductivo. En el esquema transductivo, el modelo se entrena utilizando la estructura completa del grafo, permitiendo aprovechar toda la información relacional disponible mediante validación cruzada espacial. En contraste, en el esquema inductivo los modelos se entrenan utilizando subgrafos independientes, lo que permite evaluar su capacidad de generalización hacia subconjuntos de datos o regiones geográficas no incluidas en el entrenamiento, considerando una validación cruzada convencional. Esta comparación resulta particularmente relevante en aplicaciones de DSM, donde uno de los principales desafíos consiste en predecir propiedades del suelo en ubicaciones sin mediciones directas.

En este sentido, el informe constituye un precedente técnico dentro del proyecto, orientado a caracterizar la implementación de modelos basados en aprendizaje profundo que serán utilizados en etapas posteriores del análisis. Se busca identificar las principales consideraciones metodológicas necesarias para el modelado espacial del SOC, la construcción de grafos espaciales, la selección de arquitecturas de aprendizaje profundo y la evaluación de diferentes esquemas experimentales.

En conjunto, el desarrollo de estas metodologías contribuye al avance del mapeo digital de suelos basado en *Deep Learning*, proporcionando herramientas computacionales capaces de aprovechar grandes volúmenes de información geoespacial y ambiental. Su aplicación en el contexto ecuatoriano representa un paso importante hacia la generación de estimaciones más precisas y espacialmente explícitas del carbono orgánico del suelo, información fundamental para la gestión sostenible de los recursos edáficos, la planificación territorial y las estrategias de mitigación del cambio climático.

2. Objetivos

2.1. Objetivo general

Unidad de Gestión de Investigación y Proyección Social
Edificio 3, Primer piso
Av. Ladrón de Guevara E11-253 y Andalucía
Teléfono: 2976300 Ext. 1066

Desarrollar e implementar en Python arquitecturas de GNN para la estimación y predicción del SOC en Ecuador, mediante la aplicación de diferentes modelos de aprendizaje profundo basados en grafos, evaluados bajo esquemas experimentales inductivos y transductivos.

2.2. Objetivos específicos

- Construir representaciones espaciales basadas en grafos que permitan modelar la relación de proximidad entre puntos de muestreo de suelo, utilizando criterios de distancia geográfica mediante la distancia de *Haversine* y un esquema de vecindad basado en k-vecinos más cercanos, para definir las interacciones entre observaciones.
- Implementar en Python arquitecturas de aprendizaje profundo basadas en grafos, específicamente GATv2 y GraphSAGE, orientadas a problemas de regresión sobre nodos espaciales para la estimación del SOC.
- Establecer una base metodológica para el uso de Redes Neuronales de Grafos en aplicaciones de mapeo digital de suelos, proporcionando lineamientos técnicos para futuras etapas del proyecto orientadas a la generación de mapas espaciales continuos SOC.

3. Metodología

3.1. Enfoque general de modelización

En esta sección se describe el enfoque metodológico utilizado para la estimación del SOC mediante el uso de variantes de GNN. La metodología se basa en la representación de las observaciones de suelo como nodos dentro de un grafo espacial, donde las relaciones de proximidad geográfica entre puntos de muestreo se modelan mediante aristas que conectan nodos vecinos.

Este enfoque permite incorporar explícitamente la estructura espacial del problema dentro del proceso de aprendizaje profundo, permitiendo que cada observación de SOC pueda ser estimada considerando no solo sus variables predictoras, sino también la información proveniente de muestras cercanas dentro del espacio geográfico y la estructura espacial. De esta manera, el modelo puede capturar dependencias espaciales presentes en los datos, las cuales son frecuentes en propiedades edáficas debido a los procesos de

formación y variabilidad del suelo.

Sobre esta representación estructurada de los datos se implementan diferentes arquitecturas de aprendizaje profundo basadas en grafos, orientadas a problemas de regresión sobre nodos espaciales. En particular, se utilizan modelos basados en mecanismos de atención, como GATv2 y modelos basados en agregación de vecinos, como GraphSAGE. Ambas arquitecturas permiten aprender representaciones latentes de los nodos mediante la propagación y agregación de información a lo largo de la estructura del grafo.

Adicionalmente, la metodología considera la evaluación de los modelos bajo dos esquemas de aprendizaje sobre grafos: transductivo e inductivo. El esquema transductivo permite aprovechar la estructura completa del grafo durante el proceso de entrenamiento, mientras que el esquema inductivo evalúa la capacidad del modelo para generalizar hacia subconjuntos de datos que no han sido observados durante el entrenamiento. Esta comparación permite analizar el comportamiento de las arquitecturas GNN en diferentes escenarios de aprendizaje espacial. Finalmente, el proceso de modelización incluye construcción del grafo espacial, integración de un protocolo de experimentación robusto que integra: optimización de hiperparámetros mediante búsqueda Bayesiana, entrenamiento de los modelos y evaluación del desempeño predictivo, con el objetivo de identificar configuraciones de modelos que permitan mejorar la estimación del SOC a partir de información geoespacial.

3.2. Construcción del grafo espacial

La modelización mediante GNN requiere representar las observaciones del conjunto de datos mediante una estructura de grafo que capture las relaciones espaciales entre los puntos de muestreo. En este estudio, el conjunto de datos se representa como un grafo no dirigido definido como:

$$G = (V, E),$$

donde:

- $V = \{v_1, v_2, \dots, v_n\}$ es el conjunto de nodos, que representan los puntos de muestreo de suelo.
- $E \subseteq V \times V$ es el conjunto de aristas, que representan las relaciones de vecindad espacial entre los nodos.

Cada nodo v_i está asociado a un vector de características:

$$\mathbf{x}_i \in \mathbb{R}^d,$$

donde d corresponde al número de variables predictoras utilizadas por el modelo. Asimismo, cada nodo posee una variable objetivo que está asociada:

$$y_i \in \mathbb{R},$$

que corresponde al valor observado de SOC.

Definición de la proximidad espacial

Las relaciones entre nodos se definen a partir de la distancia geodésica entre coordenadas geográficas. Para ello se emplea la fórmula de *Haversine*, la cual permite calcular la distancia sobre la superficie terrestre entre dos puntos definidos por sus coordenadas de latitud y longitud.

Sean dos puntos geográficos i y j con coordenadas (ϕ_i, λ_i) y (ϕ_j, λ_j) , respectivamente. La distancia de *Haversine* se calcula como:

$$d_{ij} = 2R \times \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_j - \phi_i}{2} \right) + \cos(\phi_i) \cos(\phi_j) \sin^2 \left(\frac{\lambda_j - \lambda_i}{2} \right)} \right).$$

donde:

- R representa el radio medio de la Tierra.
- ϕ_i corresponde a la latitud en radianes del punto geográfico i .
- λ_i corresponde a la longitud en radianes del punto geográfico i .

Esta distancia se utiliza para cuantificar la proximidad espacial entre los puntos de muestreo.

Construcción del grafo mediante k -vecinos más cercanos

Una vez calculadas las distancias entre todos los puntos de muestreo, la estructura del grafo se construye mediante un esquema de k -vecinos más cercanos (k -nearest neighbors, k -NN). Bajo este criterio, cada nodo v_i se conecta con los k -nodos más próximos según la distancia geodésica.

Formalmente, el conjunto de vecinos de un nodo v_i se define como:

$$\mathcal{N}_k(i) = \{j \in V \mid j \text{ pertenece a los } k \text{ nodos más cercanos a } i\}$$

De esta forma, se define una arista entre los nodos i y j , si:

$$j \in \mathcal{N}_k(i)$$

El parámetro k determina el grado de conectividad del grafo y se considera un hiperparámetro del modelo, permitiendo evaluar diferentes configuraciones de vecindad espacial durante los experimentos.

Representación computacional del grafo

En la implementación computacional, el grafo se representa mediante la estructura *edge_index*, utilizada por la biblioteca *PyTorch Geometric*. Esta estructura define las conexiones entre nodos como pares ordenados de índices:

$$edge_index = \begin{bmatrix} i_1 & i_2 & \dots & i_m \\ j_1 & j_2 & \dots & j_m \end{bmatrix}$$

donde cada columna representa una arista que conecta el nodo i con el nodo j .

Finalmente, el conjunto de características de los nodos, la variable objetivo y la estructura de conectividad del grafo se integran en un objeto *Data*, el cual constituye la entrada utilizada por los modelos de aprendizaje profundo implementados en el presente estudio.

3.3 Arquitecturas de Redes Neuronales de Grafos

Una vez definida la estructura del grafo espacial, se implementan modelos de aprendizaje profundo sobre grafos orientados a problemas de regresión sobre nodos, con el objetivo de estimar el valor de SOC asociado a cada punto de muestreo. En este estudio se utilizan dos arquitecturas ampliamente empleadas en aprendizaje sobre grafos: GATv2 y GraphSAGE. Ambas permiten actualizar las representaciones de los nodos mediante la agregación de información proveniente de sus vecinos dentro del grafo.

En términos generales, las representaciones de los nodos se actualizan de manera iterativa a través de capas de la red. Sea $\mathbf{h}_i^{(l)}$ la representación del nodo i en la capa l , la actualización general puede expresarse como:

$$\mathbf{h}_i^{(l)} = \sigma \left(\text{AGG} \left(\{\mathbf{h}_j^{(l-1)} : j \in \mathcal{N}(i)\} \right) \right),$$

donde:

- $h_i^{(l)}$ representa la representación latente del nodo i en la capa l .
- $h_j^{(l-1)}$ corresponde a la representación del nodo vecino j en la capa anterior.
- $\mathcal{N}(i)$ es el conjunto de nodos vecinos del nodo i definido por la estructura del grafo.
- $\text{AGG}(\cdot)$ es una función de agregación que combina la información proveniente de los vecinos.
- $\sigma(\cdot)$ representa una función de activación no lineal aplicada después de la agregación.

3.3.1 Graph Attention Networks (GATv2)

Las Graph Attention Networks introducen un mecanismo de atención que permite asignar diferentes niveles de importancia a los nodos vecinos durante el proceso de agregación. En lugar de combinar las representaciones vecinas de forma uniforme, el modelo aprende coeficientes que determinan qué vecinos contribuyen en mayor medida a la actualización del nodo objetivo.

En esta arquitectura, el coeficiente de atención entre un nodo i y uno de sus vecinos j se calcula como:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_i \parallel W\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_i \parallel W\mathbf{h}_k]))}$$

donde:

- α_{ij} es el coeficiente de atención que mide la importancia del nodo j para el nodo i .
- W es una matriz de pesos entrenables que transforma las representaciones de los nodos.
- h_i y h_j son las representaciones de los nodos i y j .
- $\mathbf{a}$ es un vector de parámetros entrenables utilizado para calcular el mecanismo de atención.
- $\parallel$ representa la concatenación de vectores.
- $\text{LeakyReLU}(\cdot)$ es una función de activación utilizada para introducir no linealidad

en el cálculo de la atención.

- $\mathcal{N}(i)$ es el conjunto de vecinos del nodo i definido por la estructura del grafo.

Una vez calculados los coeficientes de atención, la actualización de la representación del nodo se realiza mediante:

$$\mathbf{h}_i^{(l)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W \mathbf{h}_j^{(l-1)} \right),$$

donde:

- $\mathbf{h}_i^{(l)}$ es la representación actualizada del nodo i en la capa l .
- α_{ij} pondera la contribución del nodo vecino j .
- $W \mathbf{h}_j^{(l-1)}$ corresponde a la transformación lineal de la representación del vecino.
- $\sigma(\cdot)$ representa la función de activación aplicada a la salida de la capa.

En la implementación desarrollada en este estudio se utiliza la variante GATv2, implementada mediante la capa GATv2Conv de la biblioteca PyTorch Geometric, la cual permite modelar interacciones entre nodos mediante mecanismos de atención multi-cabeza (multi-head attention).

3.3.2 Graph Sample and Aggregate (GraphSAGE)

La arquitectura GraphSAGE propone un enfoque basado en la agregación explícita de la información proveniente de los vecinos de cada nodo. A diferencia de los modelos de atención, GraphSAGE utiliza funciones de agregación que combinan las representaciones vecinas mediante operaciones estadísticas, como la media o la suma.

El proceso de agregación en la capa l se define como:

$$\mathbf{h}_{\mathcal{N}(i)}^{(l)} \text{AGG}^{(l)} \left(\left\{ \mathbf{h}_j^{(l-1)} : j \in \mathcal{N}(i) \right\} \right)$$

donde:

- $\mathbf{h}_{\mathcal{N}(i)}^{(l)}$ representa la representación agregada del vecindario del nodo i .

- $AGG^{(l)}(\cdot)$ es una función de agregación aplicada a las representaciones de los vecinos.
- $\mathbf{h}_j^{(l-1)}$ corresponde a la representación del nodo vecino j en la capa anterior.
- $\mathcal{N}(i)$ representa el conjunto de vecinos del nodo i .

Posteriormente, la representación del nodo se actualiza mediante:

$$\mathbf{h}_i^{(l)} = \sigma \left(W^{(l)} \left[\mathbf{h}_i^{(l-1)} \parallel \mathbf{h}_{\mathcal{N}(i)}^{(l)} \right] \right)$$

donde:

- $W^{(l)}$ es una matriz de parámetros entrenables en la capa l .
- $\mathbf{h}_i^{(l-1)}$ corresponde a la representación previa del nodo i .
- $\mathbf{h}_{\mathcal{N}(i)}^{(l)}$ es la información agregada de los vecinos.
- $\parallel$ indica la concatenación de vectores.
- $\sigma(\cdot)$ es una función de activación no lineal.

En la implementación práctica del presente estudio, esta arquitectura se implementa mediante la capa *SAGEConv* de la biblioteca *PyTorch Geometric*, que realiza la agregación de información entre nodos vecinos definida en el grafo.

3.4. Protocolo experimental

Con el objetivo de evaluar el desempeño de las arquitecturas en su entrenamiento y prueba, se definió un protocolo experimental que integra estrategias de partición de datos, validación cruzada, optimización de hiperparámetros y entrenamiento de modelos. Este protocolo se implementó en Python, utilizando principalmente las bibliotecas *PyTorch*, *PyTorch Geometric*, *Scikit-learn* y *Hyperopt*, lo que permite estructurar un flujo reproducible de experimentación para modelos de aprendizaje profundo sobre grafos. El protocolo permite comparar el desempeño de los modelos bajo dos esquemas de aprendizaje sobre grafos: inductivo y transductivo, los cuales difieren en la forma en que la estructura del grafo se utiliza durante el entrenamiento y la evaluación.

3.4.1 Esquema de aprendizaje inductivo

En el esquema inductivo, el modelo se entrena utilizando un subconjunto de observaciones

y posteriormente se evalúa sobre datos que no han sido utilizados durante el entrenamiento. En este enfoque, los grafos utilizados para entrenamiento y evaluación se construyen de manera independiente, lo que permite evaluar la capacidad del modelo para generalizar hacia nuevos nodos.

Sea $D = \{(x_i, y_i)\}_{i=1}^n$ el conjunto completo de observaciones, donde x_i representa el vector de variables predictoras asociado al nodo i y y_i corresponde al valor observado de SOC.

Inicialmente se realiza una partición del conjunto de datos mediante la función *train_test_split* de la biblioteca *Scikit-learn*, obteniendo:

$$D = D_{train} \cup D_{test},$$

$$D_{train} \cap D_{test} = \emptyset$$

donde:

- D corresponde al conjunto completo de Datos.
- D_{train} corresponde al conjunto utilizado para el entrenamiento y la optimización de hiperparámetros.
- D_{test} corresponde al conjunto reservado para la evaluación final del modelo.

A partir de estos subconjuntos se construyen grafos independientes utilizando el método de k -vecinos más cercanos descrito previamente, generando objetos *Data* de *PyTorch Geometric* que contienen las características de los nodos, la variable objetivo y la estructura de conectividad del grafo.

Validación cruzada en el esquema inductivo

La selección de hiperparámetros se realiza mediante validación cruzada K -Fold, implementada con la clase *KFold* de *Scikit-learn*.

Este procedimiento divide el conjunto de entrenamiento en K -subconjuntos disjuntos:

$$D_{train} = \bigcup_{k=1}^K D_k$$

En cada iteración:

- uno de los subconjuntos D_k se utiliza como conjunto de validación,

- los restantes $K - 1$ subconjuntos se utilizan para entrenar el modelo.

Para cada partición se construyen grafos independientes para los conjuntos de entrenamiento y validación mediante la función *build_pyg_data*. Esto garantiza que el modelo se entrene y evalúe sobre subgrafos distintos, permitiendo analizar su capacidad de generalización hacia nodos no observados durante el entrenamiento.

Una vez identificada la configuración óptima de hiperparámetros, el modelo se entrena utilizando todo el conjunto D_{train} y posteriormente se evalúa sobre el conjunto D_{test}

3.4.2 Esquema de aprendizaje transductivo

En el esquema transductivo, el modelo se entrena utilizando la estructura completa del grafo, es decir, todos los nodos del conjunto de datos están presentes en el grafo durante el entrenamiento.

Los nodos se dividen en subconjuntos de entrenamiento y prueba:

$$V = V_{train} \cup V_{test}, V_{train} \cap V_{test} = \emptyset,$$

donde:

- V_{train} corresponde a los nodos utilizados para actualizar los parámetros del modelo.
- V_{test} corresponde a los nodos reservados para evaluar el desempeño del modelo.

En este esquema, los nodos de prueba permanecen dentro del grafo global, permitiendo que el modelo utilice información estructural proveniente de nodos vecinos durante el aprendizaje.

Validación cruzada en el esquema inductivo

Para la selección de hiperparámetros se implementa un esquema de validación cruzada espacial, diseñado para reducir el efecto de autocorrelación espacial entre las observaciones de entrenamiento y validación.

En este procedimiento, los nodos del conjunto de entrenamiento se agrupan mediante el algoritmo *K-Means*, utilizando sus coordenadas geográficas (latitud y longitud). Este proceso se implementa mediante la clase *KMeans* de la biblioteca *Scikit-learn*.

Sea $C = \{C_1, C_2, \dots, C_K\}$ el conjunto de *clusters* espaciales obtenidos. En cada iteración:

$$V_{train} = \bigcup_{i \neq k} C_i$$

$$V_{val} = C_k$$

donde C_k corresponde al *cluster* utilizado como conjunto de validación.

El grafo global se construye mediante la función *build_pyg_data_transductive*, que genera un objeto *Data* de *PyTorch Geometric* que incluye:

- las características de todos los nodos,
- la estructura completa del grafo,
- máscaras que indican los nodos de entrenamiento y validación.

Este enfoque permite evaluar la capacidad del modelo para generalizar hacia regiones geográficas distintas dentro del mismo grafo.

3.4.3 Preprocesamiento de variables

Antes del entrenamiento de los modelos se realiza un proceso de preprocesamiento de las variables, implementado mediante herramientas de la biblioteca *Scikit-learn*. Primero se realiza la imputación de valores faltantes mediante la clase *SimpleImputer*, utilizando la mediana como estrategia de imputación.

Posteriormente, las variables predictoras se normalizan utilizando diferentes esquemas de escalamiento considerados como hiperparámetros del modelo:

- **StandardScaler**
- **MinMaxScaler**
- **RobustScaler**

La variable objetivo (SOC) se transforma mediante una transformación logarítmica:

$$y^* = \log(1 + y)$$

donde y corresponde al valor original de SOC.

Posteriormente, los valores transformados se escalan utilizando *StandardScaler* con los datos de entrenamiento para evitar algún tipo de fuga de datos o de contaminación con la información de los datos prueba. Una vez obtenidas las predicciones del modelo, se aplica la transformación inversa:

$$\hat{y} = \exp(\hat{y}^*) - 1$$

y los valores negativos se restringen a cero mediante truncamiento, debió a que la estimación del SOC debe estar dentro de su propio dominio (valores mayores iguales a cero).

3.4.4 Optimización bayesiana de hiperparámetros

La selección de hiperparámetros dentro del protocolo experimental en el entrenamiento se realiza mediante optimización bayesiana, utilizando la biblioteca *Hyperopt*.

Sea $f(\theta)$ función objetivo que mide el error de predicción del modelo para una configuración de hiperparámetros θ . El problema de optimización se define como:

$$\theta^* = \arg \min_{\theta \in \Theta} f(\theta)$$

donde:

- θ representa el conjunto de hiperparámetros del modelo.
- Θ corresponde al espacio de búsqueda de hiperparametros.
- $f(\theta)$ es el error promedio obtenido durante la validación cruzada.

La optimización se realiza mediante el algoritmo *Tree-structured Parzen Estimator* (TPE), implementado en *Hyperopt*. Este algoritmo modela dos distribuciones de probabilidad:

$$l(\theta) = p(\theta \mid f(\theta) < f^*)$$

$$g(\theta) = p(\theta \mid f(\theta) \geq f^*)$$

donde:

- $l(\theta)$ corresponde a configuraciones con buen desempeño,
- $g(\theta)$ corresponde a configuraciones con peor desempeño.

El algoritmo selecciona nuevas configuraciones maximizando:

$$\frac{l(\theta)}{g(\theta)}$$

lo que dirige la búsqueda hacia regiones prometedoras del espacio de hiperparámetros.

3.4.5 Espacio de búsqueda de hiperparámetros

La Tabla 1 presenta el espacio de búsqueda definido para los principales hiperparámetros considerados en la optimización de GraphSAGE y GATv2. La selección de estos hiperparámetros responde a su influencia directa en la capacidad de representación, estabilidad del entrenamiento y desempeño predictivo de los modelos sobre datos estructurados en grafos.

Hiperparámetro	Espacio de búsqueda	Función en GraphSAGE	Función en GATv2
<i>normalizer</i>	$\{standard, minmax, robust\}$	Normaliza las características de nodos antes de la agregación vecinal.	Normaliza las características antes del mecanismo de atención.
<i>k_graph</i>	$\{3, 5, 8, 10, 12\}$	Número de vecinos muestreados por nodo en cada capa.	Número de vecinos considerados para el cálculo de coeficientes de atención.
<i>hidden_dim</i>	$\{32, 64, 128\}$	Dimensión de las representaciones ocultas tras cada agregación.	Dimensión del embedding por cabeza de atención.
<i>num_layers</i>	$\{2, 3\}$	Número de iteraciones de agregación (alcance k-hop).	Número de capas de atención apiladas.
<i>dropout</i>	$U(0.1, 0.5)$	Regulariza las representaciones ocultas tras agregación.	Se aplica a características y/o coeficientes de atención.
<i>learning_rate</i>	$LogU(10^{-4}, 5 \times 10^{-3})$	Controla la magnitud de actualización de pesos.	Controla la actualización de pesos y parámetros de atención.
<i>weight_decay</i>	$LogU(10^{-4}, 10^{-1})$	Regularización L2 sobre los pesos del modelo.	Regularización L2 sobre pesos y proyecciones de atención.
<i>residual</i>	$\{0, 1\}$	Conexiones residuales entre capas para	Conexiones residuales entre capas de atención.

		estabilizar el entrenamiento.	
<i>attention_heads</i>	{2, 4, 8}	-	Número de cabezas de atención asociadas con el mecanismo de atención en las capas GAT/GATv2

Tabla 1. Espacio de búsqueda y función de hiperparámetros en GraphSAGE y GATv2

En primer lugar, el hiperparámetro *normalizer* determina el método de normalización aplicado a las características de los nodos antes del proceso de propagación de información. En GraphSAGE, esta normalización impacta directamente la estabilidad de la agregación vecinal, mientras que en GATv2 condiciona el comportamiento del mecanismo de atención al afectar la escala relativa de las características de entrada, lo cual puede modificar la distribución de los coeficientes atencionales.

El parámetro *k_graph* controla el número de vecinos considerados en cada capa. En GraphSAGE, define explícitamente el tamaño del muestreo vecinal utilizado para aproximar la agregación local, influyendo en el costo computacional y en el alcance estructural capturado. En GATv2, aunque no se basa estrictamente en muestreo, determina el conjunto de nodos sobre el cual se calculan los coeficientes de atención, afectando la densidad de interacción y la complejidad del modelo.

Por su parte, *hidden_dim* establece la dimensionalidad de las representaciones internas. En GraphSAGE, corresponde al tamaño del *embedding* generado tras cada operación de agregación y transformación lineal. En GATv2, define la dimensión del *embedding* aprendido por cada cabeza de atención, impactando la capacidad expresiva del modelo y el número total de parámetros entrenables.

El hiperparámetro *num_layers* determina la profundidad de la red. En GraphSAGE, cada capa adicional incrementa el alcance de agregación a un salto (*k-hop*) adicional en el grafo, permitiendo capturar dependencias estructurales de mayor orden. En GATv2, representa el número de capas de atención apiladas, lo cual amplía la capacidad de modelar interacciones complejas, aunque puede incrementar el riesgo de sobre-suavizado (*over-smoothing*).

En términos de regularización, *dropout* se aplica para mitigar el sobreajuste. En GraphSAGE actúa sobre las representaciones ocultas posteriores a la agregación, mientras que en

GATv2 puede aplicarse tanto a las características transformadas como a los coeficientes de atención, introduciendo ruido estocástico que favorece la generalización.

El parámetro *learning_rate*, muestreado en escala logarítmica, controla la magnitud de actualización de los pesos durante la optimización. Su correcta calibración resulta crítica en ambas arquitecturas, particularmente en GATv2, donde intervienen además parámetros asociados al mecanismo de atención.

De forma complementaria, *weight_decay* implementa regularización *L2* sobre los parámetros del modelo. En GraphSAGE penaliza principalmente los pesos de transformación lineal, mientras que en GATv2 también afecta las proyecciones empleadas para calcular la atención, contribuyendo a evitar el crecimiento excesivo de los coeficientes. Cabe mencionar que el modelo GATv2 cuenta con el hiperparámetro *attention_heads* que cuantifica el número de cabezas de atención asociadas con el mecanismo de atención en las capas GAT/GATv2 dentro de la arquitectura, logrando ponderar para cada nodo el peso que tiene la información de sus vecinos cercanos a lo largo de todo el grafo, este mecanismo no se encuentra incorporados en GraphSAGE.

Finalmente, el hiperparámetro residual habilita o deshabilita conexiones residuales entre capas. Estas conexiones favorecen la estabilidad del entrenamiento y la preservación de información inicial en arquitecturas profundas. En GraphSAGE facilitan el flujo de gradientes a través de múltiples etapas de agregación, mientras que en GATv2 estabilizan la propagación de señales en presencia de múltiples capas de atención.

En conjunto, el espacio de búsqueda definido en la Tabla 1 permite explorar configuraciones que equilibran capacidad expresiva, estabilidad numérica y eficiencia computacional, garantizando una evaluación rigurosa y sistemática del desempeño de ambas arquitecturas sobre datos relacionales complejos. Durante la optimización, el algoritmo evalúa diferentes configuraciones entrenando el modelo y calculando el error promedio en los conjuntos de validación.

3.4.6 Estrategia de entrenamiento

El entrenamiento de los modelos se realiza utilizando el optimizador Adam (*Adaptive Moment Estimation*), el cual actualiza los parámetros del modelo mediante un esquema de descenso del gradiente adaptativo. Este optimizador combina las ventajas de los métodos de optimización basados en momento y de los métodos de escalamiento adaptativo de la tasa de aprendizaje, permitiendo una convergencia más estable en modelos de aprendizaje

Durante el entrenamiento, el optimizador ajusta los parámetros θ del modelo minimizando una función de pérdida definida sobre las predicciones generadas por la red. El proceso de actualización de los parámetros se realiza iterativamente de acuerdo con:

$$\theta_{t+1} = \theta_t - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

donde:

- θ_t representa el vector de parámetros del modelo en la iteración t ,
- η es la tasa de aprendizaje,
- $\hat{m}_t$ y $\hat{v}_t$ corresponden a las estimaciones corregidas del primer y segundo momento del gradiente,
- ϵ es un término de estabilidad numérica (que puede ser dado o puede fijarse como un valor entre 0 y 1).

Función de pérdida

Para el entrenamiento de los modelos se utiliza la pérdida Huber, también conocida como Smooth L1 Loss, la cual combina propiedades de las funciones de pérdida *Mean Square Error* y *Mean Absolute Error*, permitiendo reducir la influencia de valores atípicos en el proceso de aprendizaje.

La función de pérdida se define como:

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{si } |y - \hat{y}| \leq \delta \\ \delta(|y - \hat{y}| - \frac{1}{2}\delta) & \text{si } |y - \hat{y}| > \delta \end{cases}$$

donde:

- y corresponde al valor observado de SOC,
- $\hat{y}$ corresponde al valor predicho por el modelo,
- δ es un parámetro que controla el punto de transición entre el comportamiento cuadrático y lineal de la función de pérdida.

Cuando el error de predicción es pequeño, la pérdida se comporta como un error cuadrático, favoreciendo una convergencia suave. En cambio, cuando el error es grande, la pérdida adopta un comportamiento lineal, reduciendo la sensibilidad a valores extremos.

Regularización del modelo

Con el objetivo de mejorar la capacidad de generalización de los modelos y reducir el riesgo de sobreajuste, durante el entrenamiento se incorporan diferentes mecanismos de regularización.

Entre ellos se incluyen:

- *Dropout*, aplicado en las capas ocultas del modelo, que consiste en desactivar aleatoriamente una fracción de neuronas durante cada iteración de entrenamiento.
- *DropEdge*, técnica utilizada en redes neuronales de grafos que elimina aleatoriamente un subconjunto de aristas del grafo durante el entrenamiento. Este mecanismo introduce ruido estructural controlado en el grafo, favoreciendo la robustez del modelo frente a variaciones en la conectividad.
- Regularización *L2*, implementada mediante el parámetro *weight decay* del optimizador, que penaliza valores grandes de los parámetros del modelo.
- *Gradient clipping*, que limita la magnitud del gradiente durante el proceso de retropropagación, evitando problemas de inestabilidad numérica asociados a gradientes explosivos.

Early stopping

Además, se implementa un mecanismo de early stopping, que detiene el proceso de entrenamiento cuando el error de validación no mejora durante un número determinado de iteraciones consecutivas.

Este criterio permite evitar el sobreajuste del modelo, ya que impide que el entrenamiento continúe una vez que el modelo ha alcanzado su mejor desempeño sobre los datos de validación.

En la implementación desarrollada, el entrenamiento se realiza durante un número máximo de épocas predefinido, pero puede finalizar anticipadamente si se activa el criterio de parada temprana.

3.4.7 Métricas de evaluación

Para evaluar el desempeño predictivo de los modelos propuestos, se emplearon cuatro

métricas complementarias ampliamente utilizadas en problemas de regresión: *Root Mean Squared Error* (RMSE), *Mean Absolute Error* (MAE), coeficiente de determinación (R^2) y *Mean Absolute Percentage Error* (MAPE). Estas métricas permiten cuantificar la precisión de las predicciones desde diferentes perspectivas, proporcionando una evaluación integral del comportamiento de los modelos.

El *Root Mean Squared Error* (RMSE) mide la magnitud promedio del error cuadrático entre los valores observados y los valores predichos:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

donde:

- y_i corresponde al valor observado de SOC para la observación i .
- $\hat{y}_i$ corresponde al valor predicho por el modelo.
- n es el número total de observaciones.

El RMSE penaliza fuertemente errores grandes, por lo que es especialmente sensible a predicciones alejadas del valor real. Debido a que eleva los errores al cuadrado antes de promediarlos, el RMSE penaliza de manera más severa los errores grandes, siendo particularmente sensible a predicciones significativamente alejadas del valor real.

Por otra parte, el *Mean Absolute Error* (MAE) mide el error promedio en términos absolutos:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

A diferencia del RMSE, el MAE no amplifica los errores grandes, lo que lo convierte en una medida más robusta frente a valores atípicos. Esta métrica ofrece una interpretación directa del error medio en las mismas unidades de la variable objetivo.

El coeficiente de determinación mide la proporción de la variabilidad de la variable objetivo que es explicada por el modelo:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

donde:

- $\bar{y}$ corresponde al valor promedio observado de SOC.

Valores de R^2 cercanos a 1 indican una mayor capacidad explicativa del modelo.

Finalmente, el *Mean Absolute Percentage Error* (MAPE) mide el error promedio en términos porcentuales:

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Esta métrica permite interpretar el error de predicción de forma relativa respecto al valor observado, facilitando comparaciones entre distintos conjuntos de datos o configuraciones experimentales.

En conjunto, estas métricas permiten evaluar de manera rigurosa y complementaria el desempeño predictivo de los modelos bajo diferentes configuraciones experimentales, facilitando la comparación objetiva entre las distintas arquitecturas de redes neuronales de grafos consideradas en el estudio.

3.5 Implementación computacional

La metodología descrita en las secciones anteriores fue implementada en el lenguaje de programación *Python*, utilizando bibliotecas especializadas para aprendizaje profundo, aprendizaje sobre grafos y análisis de datos.

Entre las principales bibliotecas utilizadas se encuentran:

- *PyTorch*, utilizada como *framework* principal para la implementación de modelos de aprendizaje profundo.
- *PyTorch Geometric*, que proporciona estructuras y operadores especializados para el aprendizaje sobre grafos.
- *Scikit-learn*, empleada para tareas de preprocesamiento de datos, partición de conjuntos de entrenamiento y validación, y cálculo de métricas.
- *Hyperopt*, utilizada para la optimización bayesiana de hiperparámetros mediante el algoritmo *Tree-structured Parzen Estimator*.
- *Pandas* y *NumPy*, empleadas para la manipulación y organización de los datos.

La implementación se estructura en diferentes componentes funcionales que permiten organizar el flujo de experimentación de manera modular. En primer lugar, se definen funciones para el preprocesamiento de los datos, incluyendo imputación de valores faltantes y normalización de variables. Posteriormente, se implementan funciones para la construcción del grafo espacial, donde se calcula la estructura de conectividad entre nodos a partir de la distancia geográfica y el criterio de k -vecinos más cercanos.

Las arquitecturas de redes neuronales se implementan como clases derivadas de *torch.nn.Module*, permitiendo definir explícitamente las capas de convolución sobre grafos, los mecanismos de agregación o atención, y las capas finales de regresión. Estas clases encapsulan tanto la estructura del modelo como el proceso de propagación hacia adelante (*forward pass*).

En la Figura 1, se presenta el Protocolo Experimental - Esquema Inductivo, el cual sistematiza el flujo de trabajo para la estimación del SOC mediante arquitecturas de Graph Neural Networks (GNN). El proceso inicia con la integración de datos multimodales que abarcan sensores remotos (Sentinel-1, 2 y 3), variables climáticas y atributos topográficos, los cuales se consolidan en un Data Set junto con muestras in situ georreferenciadas. Posteriormente, el esquema emplea algoritmos avanzados como GraphSAGE y GATv2 dentro de un bucle de diseño experimental de 100 iteraciones. En este ciclo, los datos se someten a una partición (80% entrenamiento, 20% prueba) y normalización, seguidos de una optimización de hiperparámetros mediante validación cruzada (*K-fold CV*). Finalmente, el modelo óptimo transfiere sus parámetros para la predicción de SOC, cuya precisión es validada a través de métricas de evaluación para determinar la eficacia del

modelado en dependencias espaciales complejas.

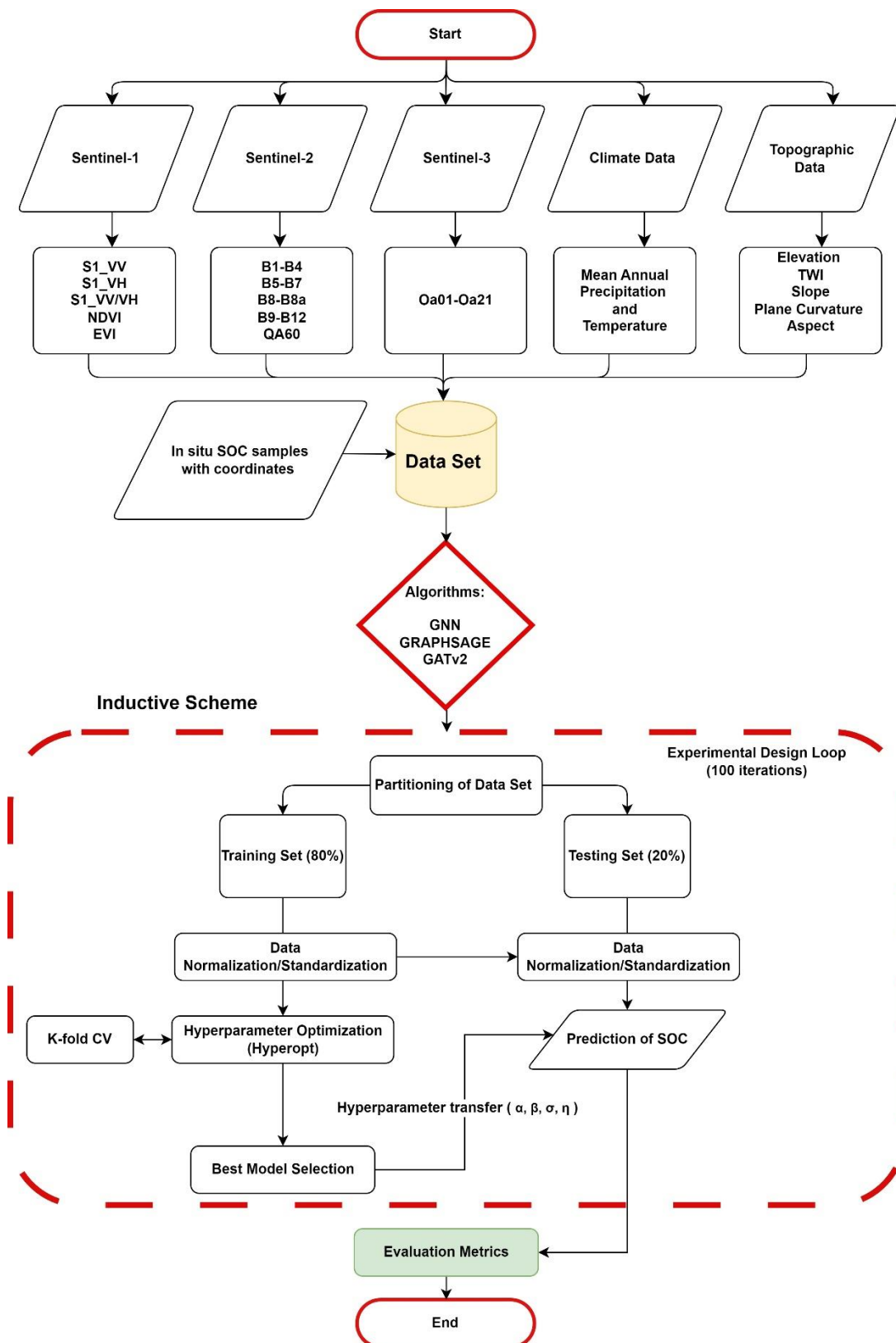


Figura 1. Protocolo Experimental - Esquema Inductivo. Fuente: Graph Neural Networks for Soil Organic Carbon

Por otra parte, en la Figura 2, se describe el Protocolo Experimental - Esquema Transductivo, el cual mantiene la fase inicial de ingesta de datos multimodales incluyendo sensores remotos (Sentinel-1, 2 y 3), datos climáticos y topográficos integrados con muestras de SOC in situ georreferenciadas. La diferencia fundamental radica en el Bucle de Diseño Experimental (100 iteraciones), donde tras la partición de datos (80% entrenamiento, 20% prueba) y su normalización, el proceso de Optimización de Hiperparámetros (*Hyperopt*) se apoya específicamente en una Validación Cruzada Espacial (*Spatial CV*) para ajustar los algoritmos de grafos como GNN, GraphSAGE y GATv2. En este esquema, se realiza una transferencia de hiperparámetros desde la selección del mejor modelo hacia la etapa de predicción, permitiendo que la red aprenda de la estructura global del grafo (incluyendo nodos no etiquetados durante el entrenamiento) para estimar el contenido de carbono. Finalmente, los resultados se consolidan mediante Métricas de Evaluación para validar la precisión del modelo en un entorno donde la información espacial es compartida entre los conjuntos de datos.

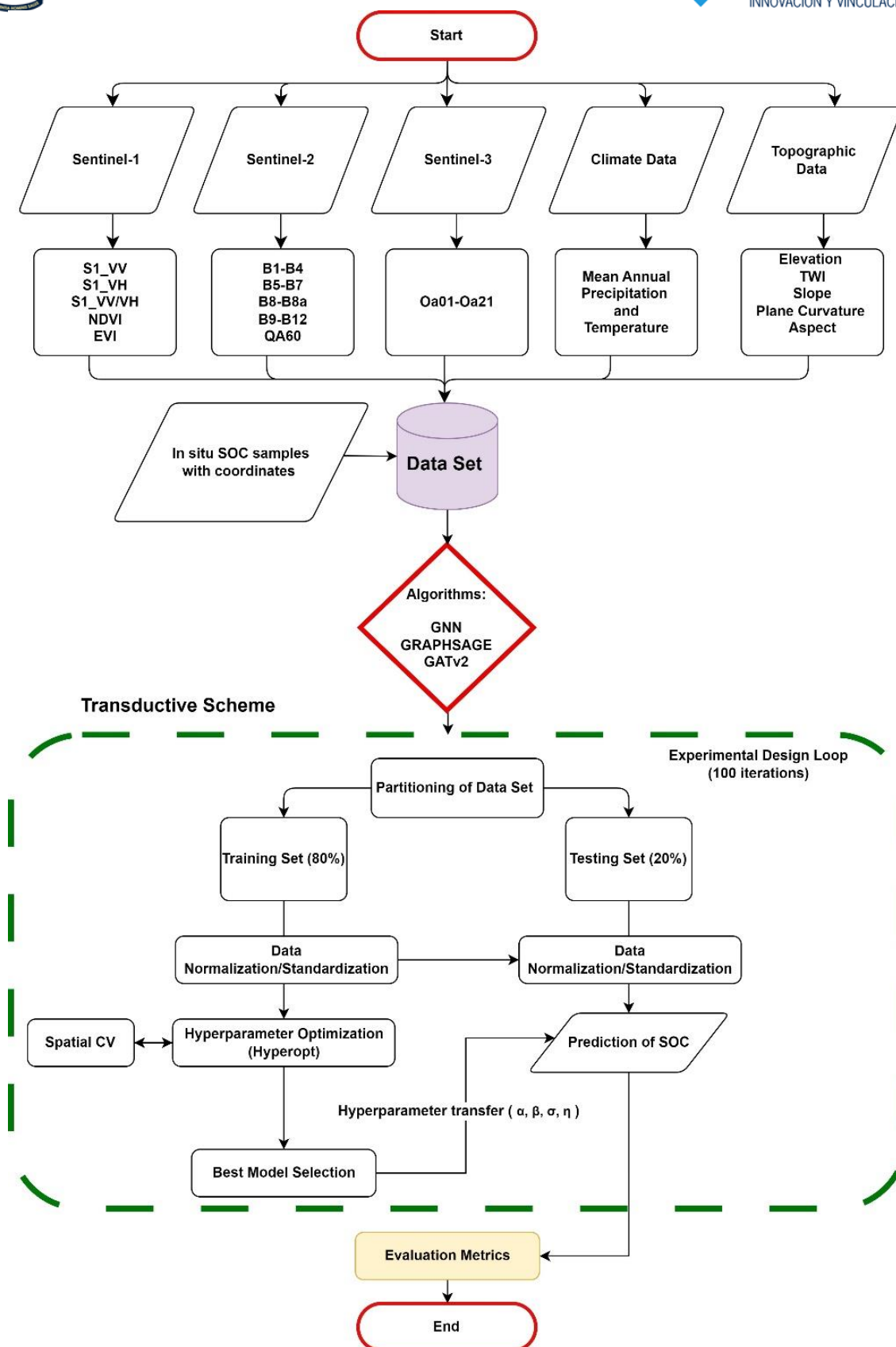


Figura 2. Protocolo Experimental - Esquema Transductivo. Fuente: Graph Neural Networks for Soil Organic Carbon Estimation: Data-Efficient Modeling of Multimodal Spatial Dependencies (2026)

El proceso de entrenamiento y evaluación se organiza mediante funciones que integran la optimización de hiperparámetros, el entrenamiento del modelo, la generación de predicciones y el cálculo de métricas de desempeño. Finalmente, los resultados obtenidos en cada experimento se almacenan en estructuras tabulares y se exportan a archivos de salida para su posterior análisis. Esta organización modular del código (Ver Anexo 1, 2, 3 y 4) permite reproducir los experimentos realizados y facilita la extensión del pipeline para evaluar nuevas arquitecturas o configuraciones experimentales en futuras etapas del proyecto.

4. Conclusiones

El presente informe describió el proceso de implementación en Python de arquitecturas avanzadas de aprendizaje profundo basadas en Redes Neuronales de Grafos (GNN) orientadas a la estimación y predicción del Carbono Orgánico del Suelo (SOC) en Ecuador. El trabajo se enfocó principalmente en el desarrollo de una metodología computacional que permite integrar información geoespacial y ambiental dentro de modelos de aprendizaje profundo capaces de representar explícitamente relaciones espaciales entre observaciones.

En primer lugar, se estableció un marco metodológico basado en la representación de los puntos de muestreo de suelo como nodos dentro de un grafo espacial, donde las relaciones de proximidad geográfica se modelan mediante aristas construidas a partir de la distancia de *Haversine* y un esquema de *k*-vecinos más cercanos. Esta representación permite incorporar la estructura espacial del problema dentro del proceso de aprendizaje, permitiendo que las estimaciones de SOC se beneficien de la información proveniente de observaciones cercanas en el espacio geográfico.

En segundo lugar, se implementaron en *Python* dos arquitecturas de aprendizaje profundo sobre grafos ampliamente utilizadas en la literatura: *Graph Attention Networks* (GATv2) y *Graph Sample and Aggregate* (GraphSAGE). Estas arquitecturas permiten aprender representaciones latentes de los nodos mediante la propagación y agregación de información entre vecinos dentro del grafo, ya sea mediante mecanismos de atención adaptativa o mediante funciones de agregación vecinal. La implementación se realizó utilizando bibliotecas especializadas como *PyTorch* y *PyTorch Geometric*, lo que permitió estructurar modelos de regresión sobre nodos espaciales de forma modular y

Adicionalmente, se definió un protocolo experimental que integra diferentes estrategias de entrenamiento y evaluación, incluyendo esquemas de aprendizaje inductivo y transductivo, validación cruzada convencional y validación cruzada espacial, optimización bayesiana de hiperparámetros mediante el algoritmo *Tree-structured Parzen Estimator* y mecanismos de regularización durante el entrenamiento. Este protocolo permite evaluar de forma sistemática diferentes configuraciones de modelos y constituye una base experimental sólida para futuras etapas de análisis. Los códigos asociados a los esquemas inductivos y transductivos para cada uno de los modelos se encuentran detallados en el Anexo 1, 2, 3 y 4 respectivamente.

Desde el punto de vista computacional, el trabajo permitió desarrollar un pipeline modular en *Python* que integra las diferentes etapas del proceso de modelización: preprocesamiento de datos, construcción del grafo espacial, definición de arquitecturas de redes neuronales, optimización de hiperparámetros, entrenamiento de modelos y almacenamiento de resultados experimentales. Esta estructura facilita la reproducibilidad de los experimentos y permite extender el sistema para incorporar nuevas arquitecturas de aprendizaje sobre grafos o nuevas fuentes de información geoespacial.

En conjunto, la implementación desarrollada establece una base metodológica y computacional para la aplicación de redes neuronales de grafos en problemas de mapeo digital de suelos. Este enfoque abre nuevas posibilidades para el análisis de propiedades edáficas utilizando información geoespacial multimodal, y constituye un paso inicial hacia el desarrollo de modelos predictivos capaces de generar estimaciones espaciales continuas del carbono orgánico del suelo en el territorio ecuatoriano.

6. Bibliografía

- Bergstra, J., Yamins, D., & Cox, D. D. (2013). Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. *Proceedings of the 30th International Conference on Machine Learning*, 115–123.
- Brody, S., Alon, U., & Yahav, E. (2022). How attentive are graph attention networks? *International Conference on Learning Representations*.
- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *International Conference on Learning Representations Workshop on Representation Learning on Graphs and Manifolds*.

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- McBratney, A. B., Mendonça Santos, M. L., & Minasny, B. (2003). On digital soil mapping. *Geoderma*, 117(1–2), 3–52. [https://doi.org/10.1016/S0016-7061\(03\)00223-4](https://doi.org/10.1016/S0016-7061(03)00223-4)
- Minasny, B., & McBratney, A. B. (2016). Digital soil mapping: A brief history and some lessons. *Geoderma*, 264, 301–311. <https://doi.org/10.1016/j.geoderma.2015.10.025>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. *International Conference on Learning Representations*.

7. Anexos

Anexo 1. Código de Python de implementación de GraphSAGE Esquema Inductivo

[Anexo 1. Código de Python de implementación de GraphSAGE Esquema Inductivo.ipynb](#)

Anexo 2. Código de Python de implementación de GraphSAGE Esquema Transductivo

[Anexo 2. Código de Python de implementación de GraphSAGE Esquema Transductivo.ipynb](#)

Anexo 3. Código de Python de implementación GATv2 Esquema Inductivo

[Anexo 3. Código de Python de implementación GATv2 Esquema Inductivo.ipynb](#)

Anexo 4. Código de Python de implementación de GATv2 Esquema Transductivo

[Anexo 4. Código de Python de implementación de GATv2 Esquema Transductivo.ipynb](#)

Elaborado por:
Nombre: Mateo Xavier Soliz Villafuerte
Servidor Público 5

Aprobado por:
Nombre: David Andrés Donoso Vargas
Director del Proyecto PIS-24-09